Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session 2025-2026

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Name and Signature of Student and Date
(Signature and Date in Handwritten)

Session	**2025-26 (ODD)**	**Course Name**	**Operating System Lab**
Semester	**5**	**Course Code**	**23IOT1504**
Roll No	**64**	**Name of Student**	**Sheikh Wasimuddin**

Practical Number	**05**
Course Outcome	1. Understand Computer System Configuration and Simulate system resources efficiently using Linux Commands (CO1) 2. Analyse operating system functionalities utilizing system calls, thread programming and process scheduling algorithms (CO2) 3. Apply Synchronization primitives to implement a Deadlock-free solution(CO3) 4. Simulate Disk scheduling, Memory allocation, File allocation, page replacement algorithms (CO4)
Aim	Simulate Banker's Deadlock detection algorithm.
Theory	In an operating system, deadlock refers to a situation where a set of processes cannot proceed because each is waiting for resources held by the others. To identify such situations, a deadlock detection algorithm is applied. This method observes how resources are allocated and requested in the system using three data structures: the Available vector (resources currently free), the Allocation matrix (resources already assigned to processes), and the Request matrix (remaining needs of processes). The algorithm begins by setting a Work vector equal to the available resources and marking all processes as unfinished. It then searches for a process whose pending requests can be satisfied with the current Work. If found, the process is assumed to complete, its allocated resources are added back to Work, and it is marked as finished. This cycle continues until either all processes finish, meaning the system is safe, or some processes remain incomplete, which signals a deadlock. Other techniques like the Wait-for Graph, Resource Allocation Graph, and Banker's Algorithm can also be used for detection. Thus, deadlock detection provides a

	systematic way for the operating system to discover and handle deadlocks when they arise.
Procedure and Execution	Code:

```
//sheikh wasimuddin_OS_PRACTICAL NO_5
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int allocation[n][m], max[n][m], need[n][m], available[m];
    bool finish[n];
    int safeSeq[n];
    int count = 0;

    printf("\nEnter Allocation Matrix (%d x %d):\n", n, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &allocation[i][j]);

    printf("\nEnter Max Matrix (%d x %d):\n", n, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("\nEnter Available Resources (%d values):\n", m);
    for (int j = 0; j < m; j++)
        scanf("%d", &available[j]);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            need[i][j] = max[i][j] - allocation[i][j];


    for (int i = 0; i < n; i++) finish[i] = false;
```

```
    while (count < n) {
        bool found = false;
        for (int i = 0; i < n; i++) {
            if (!finish[i]) {
                bool possible = true;
                for (int j = 0; j < m; j++) {
                    if (need[i][j] > available[j]) {
                        possible = false;
                        break;
                    }
                }
                if (possible) {

                    for (int j = 0; j < m; j++)
                        available[j] += allocation[i][j];
                    safeSeq[count++] = i;
                    finish[i] = true;
                    found = true;
                }
            }
        }
        if (!found) {
            printf("\nSystem is in DEADLOCK (Unsafe State).\n");
            return 0;
        }
    }

    printf("\nSystem is in a SAFE state.\nSafe Sequence: ");
    for (int i = 0; i < n; i++)
        printf("P%d ", safeSeq[i]);
    printf("\n");

    return 0;
}
```

Output:

main.c

```c
//sheikh wasimuddin_OS_PRACTICAL NO_5
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int allocation[n][m], max[n][m], need[n][m], available[m];
    bool finish[n];
    int safeSeq[n];
    int count = 0;

    printf("\nEnter Allocation Matrix (%d x %d):\n", n, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &allocation[i][j]);

    printf("\nEnter Max Matrix (%d x %d):\n", n, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("\nEnter Available Resources (%d values):\n", m);
    for (int j = 0; j < m; j++)
        scanf("%d", &available[j]);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            need[i][j] = max[i][j] - allocation[i][j];
```

```
Enter number of processS
Enter number of resources: 3

Enter Allocation Matrix (5 x 3):
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix (5 x 3):
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources (3 values):
3 3 2

System is in a SAFE state.
Safe Sequence: P1 P3 P4 P0 P2


...Program finished with exit code 0
Press ENTER to exit conso
le.
```

Output Analysis	The output of the deadlock detection algorithm tells us whether the system is in a safe state or in a deadlocked state. If the algorithm is able to find a sequence in which all processes can finish execution, then the system is declared safe and the safe sequence is printed, for example: Safe sequence: P1 P3 P4 P0 P2. This means all processes can complete without any deadlock. On the other hand, if during the checking process no unfinished process can have its request satisfied with the current available resources, then some processes remain unfinished and the algorithm concludes that the system is in deadlock. In that case, the processes marked as unfinished are considered deadlocked, and the output may display a message like "System is in DEADLOCK (Unsafe State)". Thus, the output directly indicates whether resource allocation in the system is safe or has led to deadlock.
Conclusion	In conclusion, the deadlock detection algorithm is an essential mechanism in operating systems that helps identify whether processes are stuck in a circular wait for resources. By using data structures like the Available vector, Allocation matrix, and Request matrix, the algorithm simulates process execution to check if all processes can complete. If all processes finish successfully, the system is in a safe state; otherwise, the unfinished processes are considered deadlocked. This approach ensures that deadlocks can be discovered and appropriate recovery

	actions, such as resource preemption or process termination, can be applied to maintain system stability and efficiency.
Plag Report (Similarity index < 12%)	
Date	22/09/2025